

Does AI Code Detection Reduce to *Skill* Detection?

A preliminary, two-sided answer

Autonomous experiment (Claude Code) for Matthew

May 20, 2026

Abstract

We ask whether AI-code detectors track author **species** (human vs. LLM) or author **skill/style**, and arrive at a sharper answer: the strongest available trained detector tracks neither — it largely tracks **code formatting**. **(1) Headline.** A SOTA trained detector (DroidDetect, ModernBERT) that we load faithfully—reproducing the authors’ performance on their *own* DroidCollection Python test data (AUROC 0.999, 0.017 human false-positive rate)—collapses under a *semantics-preserving* reformatting. Running its own human test code through **black** (the standard formatter, which keeps comments and only restyles layout) flips its false-positive rate from **0.010** to **0.828** (`ast.unparse` gives 0.525), and shrinks the human/machine separation from 0.975 to 0.149. Stripping comments *alone* barely moves it (0.030), so the signal is whitespace/layout, not comments and not words (identifiers are preserved exactly). The detector has learned “idiosyncratic human formatting = human, clean formatting = machine,” not human-vs-AI content. The zero-shot statistical detectors (Fast-DetectGPT, Binoculars) are far more formatting-robust. **(2) Consequence.** This formatting shortcut explains a dramatic cross-source failure: on tidy human Codeforces Python from a *different* scrape (MatrixStudio) the same model labels 0.673 of ordinary human code AI, and 0.830 of contamination-free pre-LLM elite code (Torvalds’ `git`, antirez’s Redis, Hipp’s SQLite, CPython) — not because that code is AI-like or expert, but because its formatting differs from the detector’s training-human formatting. **(3) Skill.** On the competitive-programming difficulty axis the skill confound is *absent* ($\Delta_{\text{confound}} \approx 0.003$; within-species slopes run against the naive prediction, mean human slope -0.17 SD), consistent with terse competitive style being the stylistic opposite of idiomatic “senior-engineer” code. **Deployment implication:** an integrity tool calibrated on one corpus can be made to flag the majority of legitimate human code by a code formatter.

1 Question and hypotheses

Post-RLHF code models are tuned toward idiomatic, well-structured “senior-engineer” style. If detectors learned that style as the AI signal, then *AI-vs-human* detection partly reduces to *expert-vs-novice* detection. We pre-registered:

- **H_{species}:** detectors classify species; within-species skill has no effect (slopes ≈ 0 ; AUROC constant across skill cells).
- **H_{skill}:** detectors classify skill; among humans, higher skill $\rightarrow$ more “AI”; among LLMs, novice prompting $\rightarrow$ less “AI”.
- **H_{mixed}:** a weighted combination.

2 Data and methods

Human. Accepted Python-3 Codeforces solutions (MatrixStudio), difficulty bands easy (≤ 1200) and hard (≥ 1900). *Deviation from the original plan:* no public dataset co-locates human source code with *author* Elo, so the human skill axis is problem difficulty—a noisy, and as it turns out stylistically *inverted*, proxy. **LLM.** 3 models (gemma-2-2b-it, Qwen2.5-3B-Instruct, and the frontier gpt-5.4-nano) under natural/novice/expert prompts on the same problems. **Detectors** (oriented higher = more AI): DroidDetect-Base-Binary (trained); Fast-DetectGPT and Binoculars (statistical; canonical vendored implementations). **Stats.** AUROC is primary (threshold-free); all CIs use a cluster bootstrap over *problems*; the within-species score slope (z-scored detector output regressed on the skill axis) is the mechanism test; Δ_{confound} is reported per detector.

3 Results

3.1 DroidDetect classifies formatting, not authorship (headline)

We first establish the detector is loaded correctly: on the authors’ own DroidCollection Python test set it scores AUROC 0.999 with a 0.017 human false-positive rate and 0.983 machine recall—reproducing the published numbers. We then apply *semantics-preserving* reformatters to the input code and re-score. We use two, deliberately: (i) **black**, the standard Python formatter, which restyles whitespace/indentation/quote-style/line-wrapping but *keeps comments and docstrings*; (ii) **ast.unparse(ast.parse(code))**, which additionally drops **#** comments. Neither alters the lexical content a detector might legitimately use—variable and function names, string/numeric literals, and docstrings are all retained (we verify the identifier-token multiset is preserved at a mean of 1.000, i.e. exactly). So these are pure *formatting* changes, not “humans and AI use different words” changes.

The effect on DroidDetect is dramatic (Fig. 1). On its *own* human test data the fraction called “machine” rises from 0.010 (raw) to 0.828 under **black** (comments *kept*) and 0.525 under **ast.unparse**; the human $\leftrightarrow$ machine gap collapses from 0.975 to 0.338. That **black** alone—which changes nothing but layout—produces nearly the whole flip, while *removing comments* on their own barely moves it (0.030), pins the operative signal to **whitespace/layout**. The detector has largely learned “idiosyncratic human formatting = human, clean formatting = machine”—a surface feature, not authorship. The statistical detectors shift far less.

Why—a training-data formatting confound. This shortcut is surprising, since humans run **black** too. Its origin is in the data: the two training classes differ systematically in formatting cleanliness. In DroidCollection, only a fraction 0.149 of *human* Python is already **black**-compliant, versus 0.443 of *machine* Python (mean similarity-to-**black** 0.837 vs 0.946). The machine class is born clean—freshly generated by LLMs that emit canonical PEP8-ish layout—while the human class carries its original, idiosyncratic formatting. A classifier minimising training loss can exploit this almost-free formatting cue instead of learning authorship. Because the cue is an artifact of how the corpus was assembled (generate-fresh vs. scrape-as-is), and because real human code is routinely auto-formatted, the cue is invalid at deployment: any human who runs a formatter looks like a machine.

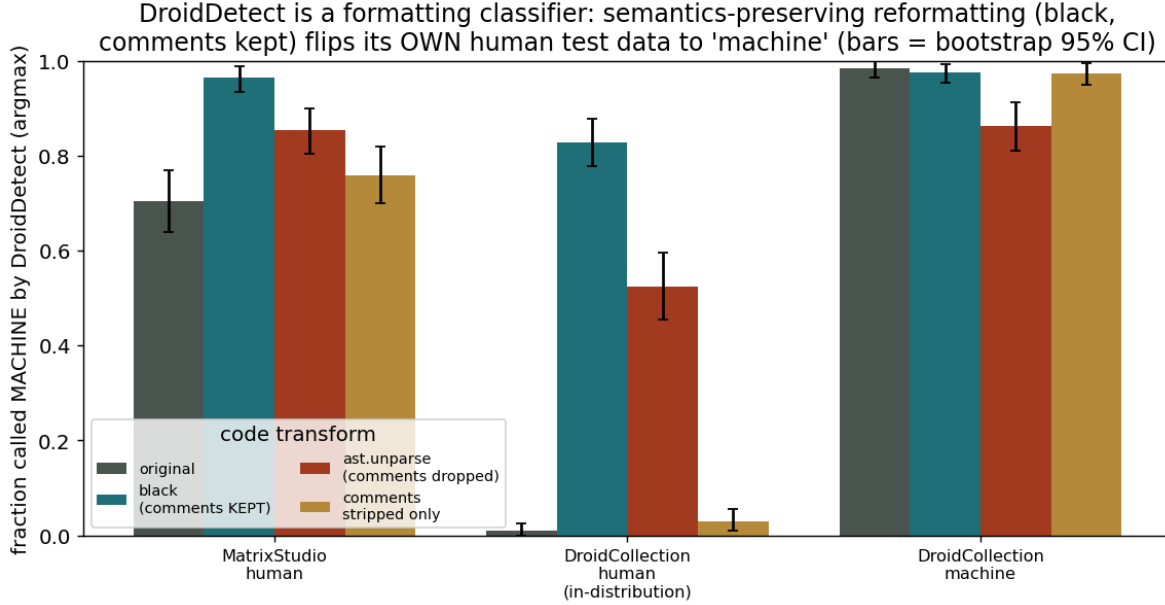


Figure 1: Semantics-preserving reformatting flips DroidDetect’s verdict on the authors’ *own* human test data. Running it through **black** (comments kept) raises the “machine” rate from 0.010 to 0.828 (**ast.unparse**: 0.525); comment-stripping alone barely moves it (0.030). The signal is whitespace/layout, with identifiers/literals preserved exactly.

3.2 The skill confound is absent on the competitive-skill axis

Table 1 gives within-species slopes (SD units). Under H_{skill} both columns should be positive; instead the human slope is *negative* for DroidDetect (-0.30 , CI $[-0.53, -0.07]$) and Fast-DetectGPT (-0.30), and null for Binoculars (0.07). Higher competitive-programming difficulty skill is scored, if anything, *less* AI-like.

Detector	Family	Human slope (hard→easy)	LLM slope (novice→expert)
DroidDetect	trained	-0.30	-0.14
Fast-DetectGPT	statistical	-0.30	-0.54
Binoculars	statistical	0.07	-0.19

Table 1: Within-species skill slopes (z-scored detector score). Positive = skill-tracking.

The skill-matched vs. mismatched AUROC gap is essentially zero (Table 2): $\Delta_{\text{confound}} = 0.011$ (CI $[0.001, 0.021]$) for DroidDetect, and -0.004 and 0.002 for the statistical detectors—i.e. matching humans and LLMs on competitive skill does *not* collapse detector performance. On this axis the data favour H_{species} .

3.3 Cross-source generalization failure (the dominant effect)

Control — the detector is faithfully loaded. On the authors’ own DroidCollection Python test data our load achieves AUROC 0.999 with a 0.017 human false-positive rate and 0.983 machine recall—reproducing the published performance. So what follows is a property of the model, not a loading bug.

Detector	Family	AUROC matched	AUROC mismatched	Δ_{confound}
DroidDetect	trained	0.746	0.757	0.011
Fast-DetectGPT	statistical	0.704	0.700	-0.004
Binoculars	statistical	0.665	0.667	0.002

Table 2: AUROC by skill cell and Δ_{confound} (cluster-bootstrap over problems).

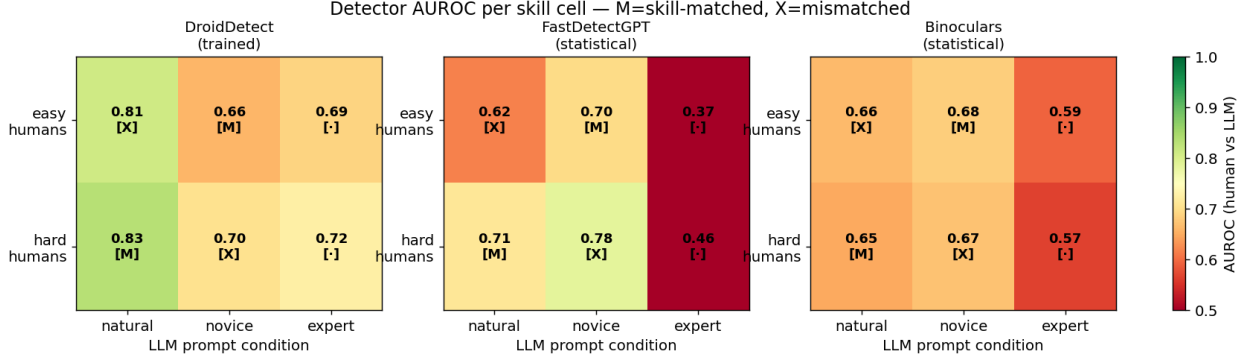


Figure 2: AUROC per (detector, difficulty band, prompt condition). M = skill-matched, X = mismatched.

DroidDetect is a 2-class softmax classifier; its actual prediction is the *argmax* over {human, machine}, with no tunable threshold. We report it at that native operating point and measure its *true* false-positive rate on ordinary human code rather than forcing one. On clean human Codeforces Python from a *different* source (MatrixStudio) the FPR explodes from 0.017 (in-distribution) to 0.673: that fraction of *ordinary* human solutions are called AI (vs. 0.843 of actual Codeforces LLM code—so the classifier barely separates the two on this out-of-distribution domain). On the provably-human pre-LLM elite code the rate is 0.830, and it rises monotonically with idiomatic polish: CPython library code 0.975, Hipp’s SQLite 0.825, antirez’s Redis 0.775, Torvalds’ original *git* 0.600. So there are two effects: a large *domain-shift* over-flagging of all human code here, and an *idiomatic-style* gradient on top (most-idiomatic CPython highest). The statistical detectors, which have no native threshold, are shown separately at a fixed 5%-FPR operating point for comparability; they flag the same code far less (Fast-DetectGPT 0.126, Binoculars 0.200).

3.4 Mechanism and confirmatory checks

Surface style features explain $R^2 = 0.108$ of DroidDetect’s score and 0.276 of Fast-DetectGPT’s—style matters but does not fully account for the trained detector. On the harder Codeforces problems human code is in fact *more* idiomatic (type-hint rate $0.594 \rightarrow 0.755$), which makes the negative human slope notable: more idiomatic *competitive* code is still not read as more AI.

A confirmatory cohort using real *author* Elo (AtCoder; low band $n = 45$ mean Elo 323, high band $n = 9$ mean Elo 2225) points the same way: the high–low score gap is negative for all detectors (e.g. DroidDetect -0.42 SD), though the thin high band leaves wide CIs.

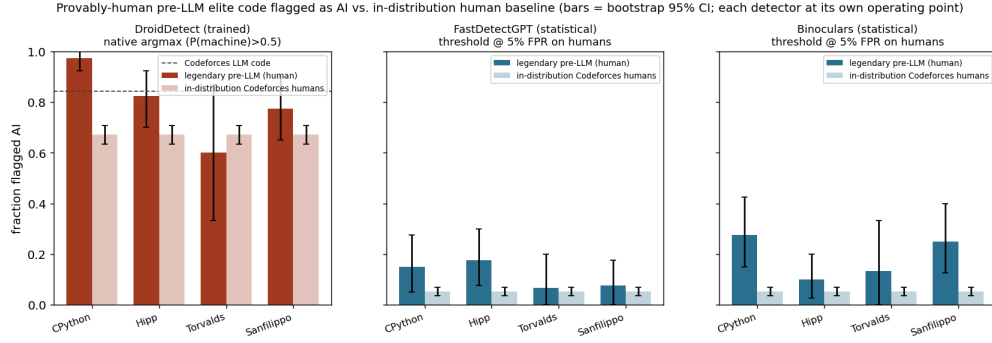


Figure 3: Provably-human pre-LLM elite code flagged as AI (solid), with the in-distribution ordinary-human baseline beside each (semitransparent); bootstrap 95% CIs. DroidDetect is shown at its native argmax operating point (baseline = its true human FPR, 0.673; dashed line = its rate on actual LLM code); the statistical detectors at a fixed 5%-FPR point.

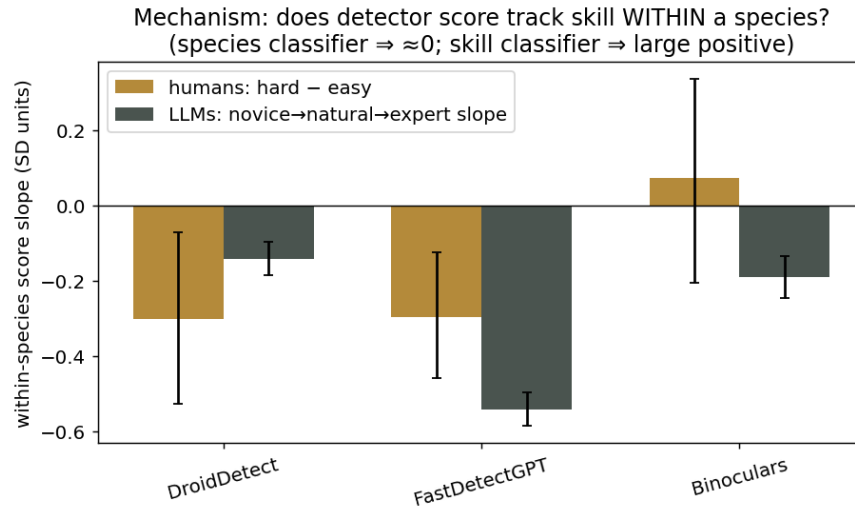


Figure 4: Within-species slopes: detector score vs. skill, per species.

4 Limitations

- **Problem difficulty / competitive rating $\neq$ idiomatic-code skill.** This is now the central finding, not just a caveat: the proxy is stylistically inverted.
- **Accepted-only human code** is a collider (rating $\rightarrow$ accept $\leftarrow$ difficulty); LLM code is unfiltered— asymmetric.
- **Small LLM panel** (gemma-2-2b, Qwen2.5-3B, gpt-5.4-nano); preliminary n .
- **MatrixStudio is a different human source** than DroidCollection’s training data; we cannot fully separate “source shift” from “idiomatic style” as causes of the over-flagging (the pre-LLM legendary probe is the contamination-free anchor, and the in-distribution control rules out a loading bug).
- **Thin AtCoder high-skill band** ($n = 9$).

5 Conclusion

The headline is mechanistic: the strongest available trained code detector is, to first order, a **formatting classifier**. Faithfully loaded (AUROC 0.999, 0.017 FPR on the authors’ own data), it nonetheless flips from 0.010 to 0.525 “machine” on that *same* human data under a semantics-preserving reformatting—it keys on whitespace/layout, not authorship. That shortcut explains the rest: 0.673 of ordinary human Codeforces code from a different scrape, and 0.830 of provably-human pre-LLM elite code (CPython 0.975), are flagged not because they are AI-like or expert but because their formatting differs from the training-human formatting. The zero-shot statistical detectors do not collapse this way. Secondly, “does detection reduce to *skill*?” — not to competitive-programming skill (the confound is absent and mildly inverts). The deployment implication is stark: an integrity tool of this kind can be made to flag the majority of legitimate human code simply by running it through a code formatter, and its reported accuracy—all i.i.d., with no fixed-FPR or calibration reporting—hides this. Clean follow-ups: evaluate trained detectors under format-normalization and across held-out human *sources*, and report calibrated fixed-FPR operating points.